

Esercitazione 5: algoritmi di ordinamento 2 - osservazioni

Gli algoritmi di ordinamento con complessità computazionale in tempo quadratica ($O(n^2)$) implementati nell'esercitazione sono bubble sort, insertion sort e selection sort; tra questi, l'insertion sort si rivela essere il più veloce, soprattutto sui vettori di piccola dimensione, grazie al minor numero di confronti effettuati in media.

Gli algoritmi di ordinamento con complessità computazionale in tempo logaritmica ($O(n \log(n))$) implementati nell'esercitazione sono merge sort e quick sort; tra questi, il quick sort si rivela essere il più veloce.

Gli algoritmi con complessità logaritmica sono quelli ottimali; tuttavia, si osserva che per vettori di piccole dimensione gli algoritmi quadratici, in particolare l'insertion sort, risultano più veloci di quelli logaritmici. La soglia n_0 al di sotto della quale gli algoritmi quadratici sono più veloci di quelli logaritmici non è universale, ma dipende dall'architettura della CPU; infatti, lo stesso codice su macchine diverse può produrre valori di n_0 diversi; anche sulla stessa macchina, il valore può variare tra un'esecuzione e un'altra a causa del rumore che può interferire con le misurazioni.

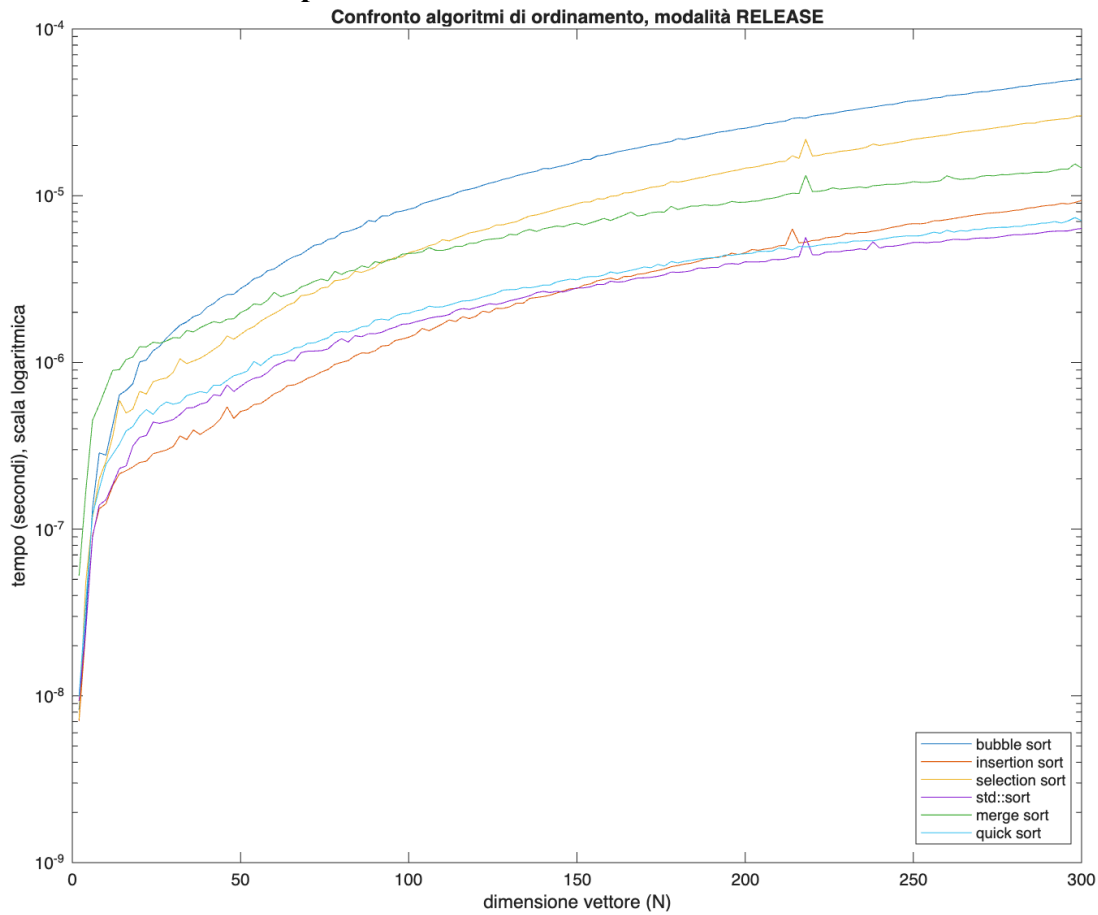
Per questo motivo, ho determinato il valore di n_0 tramite misure ripetute su una macchina Apple M2 Pro, ottenendo in modalità release $n_0 \approx 190$ e in modalità debug $n_0 \approx 50$. Al di sotto di tale soglia, i costi costanti (overhead ricorsivo, partition) del quick sort superano quelli dei confronti aggiuntivi degli algoritmi quadratici, in particolare dell'insertion sort.

Per questo motivo, è possibile implementare una versione ibrida del quick sort (quick sort hybrid) che utilizza insertion sort quando la dimensione del sottovettore corrente è inferiore della soglia n_0 individuata compilando in modalità release. La versione ibrida risulta più veloce della versione di quick sort "tradizionale" su tutto il range di dimensioni testato.

Si osserva, inoltre, che `std::sort` risulta più veloce degli altri algoritmi al di sopra di una soglia intorno ai 150 (in modalità release). Questo probabilmente è dovuto alle ottimizzazioni interne e all'uso di algoritmi ibridi di `std::sort`. Al di sopra di tale soglia, comunque, il distacco aumenta ed è notevole soprattutto se confrontato con i tempi di bubble sort.

Inoltre, si può osservare che in modalità release i tempi si riducono rispetto alla modalità debug, in particolare per gli algoritmi merge sort e quick sort. Gli algoritmi quadratici beneficiano meno delle ottimizzazioni del compilatore in modalità release rispetto a quelli logaritmici.

Grafico ottenuto compilando in modalità RELEASE



Confronto algoritmi di ordinamento, divisi per complessità, in modalità RELEASE:

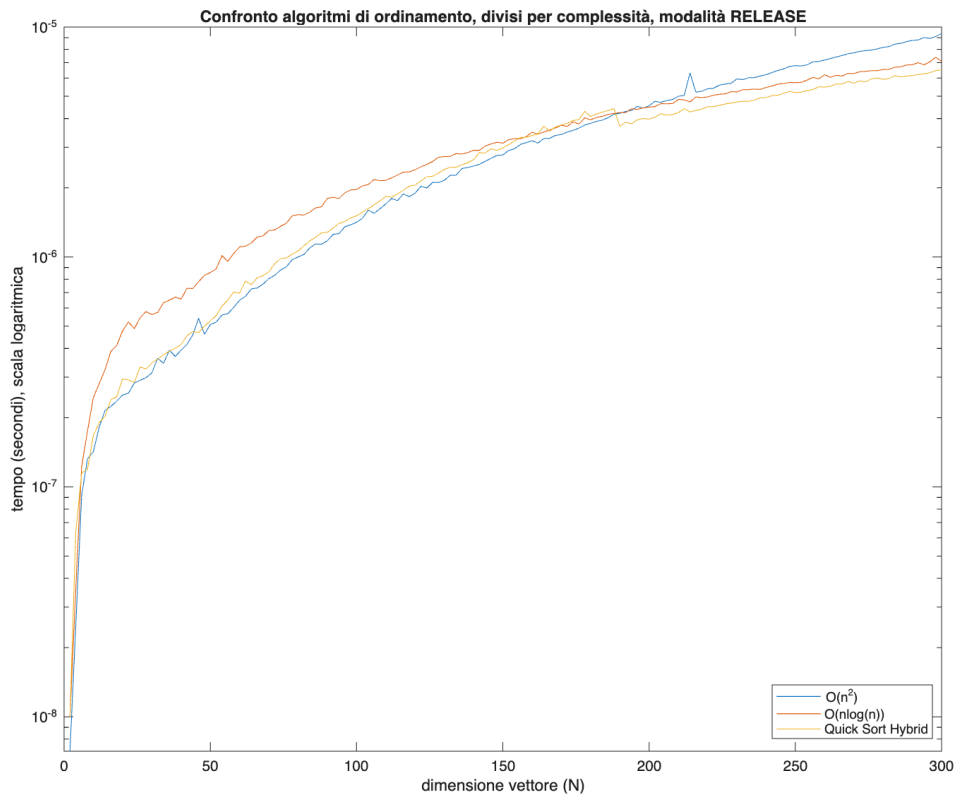


Grafico ottenuto compilando in modalità DEBUG:

